STUDENT

# S-0008-NCY

TENTAMEN

# 250422 DT141G, DT201G, DT202G Sundsvall

| | |
|---|---|
| Kurskod | -- |
| Bedömningsform | -- |
| Starttid | 22.04.2025 08:00 |
| Sluttid | 22.04.2025 13:00 |
| Bedömningsfrist | -- |
| PDF skapad | 01.12.2025 15:05 |
| Skapad av | Liselott Engelbrekts |

i

Mittuniversitetet
MID SWEDEN UNIVERSITY

# Tentamen

**DT141G Operativsystemsteori**
**DT201G, DT202G Operativsystem**

**Jimmy Åhlander**

## Instruktioner

| | |
|---|---|
| Tid | 5 timmar |
| Resurser | Dator och mus. Kalkylator, kurslitteratur, lexikon och formelsamling samt egna anteckningar, penna och sudd. Läsplatta*. |
| Poäng | 100 |
| Krav för godkänt | För godkänt betyg på tentamen fordras betyg E eller högre. |
| Preliminära betygsgränser | A 90 % B 80 % C 70 % D 60 % E 50 % |
| Allmänt | Använd stycken, punktlistor och annan textformatering där det är lämpligt när du besvarar frågor med text.<br><br>Om frågan kräver att du motiverar ditt svar, visar dina beräkningar eller ritar en figur erhålles i normalfallet inga poäng för endast svaret. Ibland är det emellertid endast en rekommendation och inte ett krav att svara på ett visst sätt.<br>Exempel på krav:<br>"*Rita en figur som styrker ditt svar*", "*Svara i bytes*", "*Visa dina beräkningar*".<br>Exempel på rekommendationer:<br>"*Rita en figur om du anser att det styrker ditt svar*", "*Tips: Skapa en tabell*".<br><br>Tentan är anonymiserad. Skriv aldrig ditt namn, personnummer eller annan identifierande information i dina svar.<br><br>Vid tekniska problem, kontakta alltid tentamensvakt. Stäng inte av eller starta om datorn utan uppsikt, om du inte redan lämnat in tentamen.<br><br>*Det är inte tillåtet att använda eller ha tillgång till mobiltelefon, surfplatta eller liknande enheter under |

tentamen.
Endast läsplatta *utan tillgång till Internet*, av följande märke är tillåten: ReMarkable.

InsperaScan används ej för denna tentamen.

**Mittuniversitetet**
**Institutionen för data- och elektroteknik (DET)**

**Jimmy Åhlander, 010 - 142 86 90**

**1(a)** Tänk dig en textfil som består av 2048 byte data, motsvarande 2048 ASCII-tecken.

Anta att filen lagras i ett filsystem av typen NTFS där du granskar filens egenskaper genom utforskaren i Windows (explorer.exe). *Anta att du har standardinställningarna (default settings) för filsystemet*.

Observera att enheterna nedan anges och ska tolkas så som de återges i utforskaren.

Hur skulle storleken presenteras för filen?

Svar: 2 KB.

Hur skulle storleken på disk presenteras för filen?

Svar: 2.048 KB.

---

Delvis rätt. 1 av 2 poäng.

**1(b)** I följande bild ses delar av innehållet i katalogen där bland annat **`file.txt`** återfinnes med hjälp av kommandot **`ls -lia`** i **Bash**. Besvara följande frågor utifrån denna information.

```
total 40
4756235 drwxr-xr-x  5 chuck staff 4096 Apr 15 00:25 .
4756236 drwxr-xr-x 18 chuck staff 4096 Apr 14 23:50 ..
4756237 -rw-r--r--  1 chuck staff 3771 Apr 15 00:20 .bashrc
4756240 -rw-r--r--  3 chuck staff   24 Apr 15 00:23 file.txt
4756241 lrwxrwxrwx  1 chuck staff    8 Apr 15 00:27 file_link -> file.txt
4756242 lrwxrwxrwx  1 chuck staff    1 Apr 15 00:29 curr_dir_link -> .
```

Hur skulle filrättigheterna för file.txt uttryckas på numeriskt (oktalt) format?

Svar: 644 .

Vem är ägare till file.txt?

Svar: chuck .

Hur många hårda länkar leder till den nuvarande katalogen (*från vad du kan se*)?

Svar: 5 stycken.

Hur många symboliska (mjuka) länkar leder till den nuvarande katalogen (*från vad du kan se*)?

Svar: 1 stycken.

Vilket nummer har inoden för föräldrakatalogen?

Svar: 4756236 .

---

Delvis rätt. 5 av 5 poäng.

**1(c)** Utgå ifrån tidigare presenterad information för att även besvara följande: Vilka av nedan angivna operationer är möjliga för ägaren av respektive fil?

**Välj ett eller flera alternativ:**

- [x] Ändra filinnehållet i **`file.txt`**.
- [ ] Exekvera **`file.txt`**, t.ex. som ett skalskript.
- [x] Skriva ut innehållet i **`file.txt`**, t.ex. med **`cat`**.
- [x] Ändra namnet på **`.bashrc`**, t.ex. med **`mv`**.
- [x] Skriva ut innehållet i den nuvarande katalogen, t.ex. med **`ls`**.
- [ ] Exekvera **`file.txt`**, t.ex. som ett skalskript, genom **`file_link`**.
- [x] Ta bort **`.bashrc`**, t.ex. med **`rm`**.

Tips:

- Filnamn inklusive filändelser säger inget om filformatet.
- 5 av alternativen är korrekta.

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

---

Rätt. 5 av 5 poäng.

**i** Sidersättningsalgoritmer tillämpas i operativsystem som nyttjar virtuellt minne och demand paging. Givet tre fysiska ramar, anta att processen $P_1$ använder sina logiska sidor i en sekvens angiven av en referenssträng där R representerar en läsoperation och W representerar en skrivoperation.

Processen $P_1$ har redan tre sidor allokerade i det fysiska minnet, se Tabell 1. $P_1$ behöver nu sidor enligt följande referenssträng: **`W4 R2 W2 R1 W0 R4 R1 W2 R0`**.

Tabell 1. Inverterad sidtabell för systemet.

| Ram | Sida | Process | SC bit | Smutsig/mod bit | Valid bit |
|---|---|---|---|---|---|
| 0 | 7 | $P_1$ | 1 | 0 | 1 |
| 1 | 1 | $P_1$ | 1 | 1 | 1 |
| 2 | 3 | $P_1$ | 0 | 0 | 1 |

Bilderna nedan representerar primärminnet uppdelat i sina ramar vid 10 olika tidpunkter: en för utgångsläget plus en för varje kommande sidreferens. Pekaren som leder till huvudet i den cirkulära kön representeras av symbolen **>**.

**2(a)** Visa hur sidorna i referenssträngen kommer att allokeras till olika ramar om algoritmen **Optimal Page Replacement (OPT)** tillämpas genom att fylla i vilken sida som är allokerad till vilken ram vid varje tidpunkt, det vill säga efter varje referens i referenssträngen.

*Endast sidnumret ska anges i respektive fält.*

*I de fall där det finns flera potentiella ramar som kan likvärdigt offras ska den ram (obs: ram, inte sida) som kommer tidigast i minnet ersättas, det vill säga helst ersätts ram 0, därefter 1, sist 2.*

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| >7 | 7 | 2 | 2 | 2 | 2 | 2 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 3 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 2 | 2 |

Delvis rätt. 0.6 av 4 poäng.

**2(b)** Hur många sidfel uppstår när OPT tillämpas i det givna scenariot?

Antal sidfel: 5 .

Fel. 0 av 1 poäng.

**2(c)** Visa hur sidorna i referenssträngen kommer att allokeras till olika ramar om algoritmen **Enhanced Second Chance (ESC)** tillämpas genom att fylla i vilken sida som är allokerad till vilken ram vid varje tidpunkt, det vill säga efter varje referens i referenssträngen.

*Endast sidnumret ska anges i respektive fält.*

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| >7 | 7 | 7 | | | | | | | |
| 1 | 1 | 2 | | | | | | | |
| 3 | 4 | 4 | | | | | | | |

Tips:

- Arbeta initialt med penna och papper för att enklare annotera och skissa på en lösning.
- Tänk på utgångsläget som presenteras i systemets sidtabell.

Delvis rätt. 0.6 av 4 poäng.

**2(d)** Hur många sidfel uppstår när ESC tillämpas i det givna scenariot?

Antal sidfel: 3 .

Fel. 0 av 1 poäng.

**2(e)** Anta i följande två deluppgifter att du får arrangera om hänvisningarna i referenssträngen ovan och använda vilken algoritm som helst för sidersättningen (etablerad variant eller egen).

Vad är det lägsta antalet sidfel som kan uppstå?

Antal sidfel: 2 .

Fel. 0 av 2 poäng.

**2(f)** Vad är det högsta antalet sidfel som kan uppstå?

Antal sidfel: 6 .

Fel. 0 av 2 poäng.

**i** Anta att du har ett exotiskt 18-bit operativsystem där kopplingen mellan logiskt och fysiskt minne sker med ett simpelt relocation register i MMU:n. Med andra ord använder systemet sammanhängande allokering och inte paging. Åtkomsttiden till primärminnet har i detta system uppmätts till 100 ns.

**3(a)** Hur mycket logiskt minne kan en process maximalt använda i detta system? *Svara i MiB. Ignorera eventuella implementationsspecifika begränsningar.*

Svar: 0.25 MiB.

Fel. 0 av 2 poäng.

**3(b)** Vi byter nu från sammanhängande allokering till paging i systemet, med sidstorleken 1 KiB. Detta får ses som främst en teoretisk exercis då paging har begränsad nytta i ett 18-bitars system. I vilket fall används ingen translation lookaside buffer (TLB).

Paging som teknik lider generellt av intern fragmentering. Hur mycket minne kan en process maximalt förlora till intern fragmentering i det här systemet? *Anta att processen nyttjar allokerade sidor i så stor utsträckning som möjligt. Svara i bytes.*

Svar: 1023 B.

Rätt. 2 av 2 poäng.

**3(c)** Hur många sidor består nu det logiska minnet av?

Svar: 256 sidor.

---

Rätt. 2 av 2 poäng.

**3(d)** Processorn i systemet har precis genererat adressen $\mathbf{1500}_{10}$. Vad blir sidnumret och förskjutningen för adressen?

Sidnummer: 1024 .

Förskjutning: 476 .

Tips:

- Du kan svara i valfri talbas av följande: 2, 10, 16 utifrån den du tycker är enklast att arbeta med. Ange dock *inte* talbasen i fälten.

---

Delvis rätt. 2 av 4 poäng.

**3(e)** Den effektiva åtkomsttiden till minnet förändras med en konstant faktor $x$ när paging används istället för sammanhängande allokering. Under antagandet att ingen translation lookaside buffer (TLB) används, vad är faktorn $x$?

**Välj ett alternativ**

- 0
- 1
- 2
- 3
- 5
- 10

*Tips: En faktor $x = 1$ innebär att den effektiva åtkomsttiden är oförändrad.*

---

Rätt. 3 av 3 poäng.

**3(f)**

Vi vill nu minimera eventuella tidsförluster genom att också introducera en TLB för minneshanteringen. Den effektiva åtkomsttiden med paging och buffert får sammanlagt **inte öka med *mer än* 25 %** jämfört med den ursprungliga åtkomsttiden till primärminnet.

Vi har fått fem olika bufferts att välja mellan för att uppfylla kravet. Snabbare och träffsäkrare TLBs kostar mer varför vi önskar välja en så billig buffert som möjligt som samtidigt uppfyller vårt krav.

Vilken är den billigaste bufferten som uppfyller kravet?

**Välj ett alternativ:**

- **`billigast:`**
  $85\ \%$ träffchans, $12\ ns$ åtkomsttid
- **`billigare:`**
  $87\ \%$ träffchans, $12\ ns$ åtkomsttid
- **`medeldyr:`**
  $85\ \%$ träffchans, $10\ ns$ åtkomsttid
- **`dyrare:`**
  $90\ \%$ träffchans, $8\ ns$ åtkomsttid
- **`dyrast:`**
  $95\ \%$ träffchans, $5\ ns$ åtkomsttid
- Ingen buffert uppfyller kravet.

---

Fel. 0 av 5 poäng.

Din tekniskt obegåvade, hypotetiska kusin har av någon spännande anledning börjat använda Ubuntu. Din kusin har "upptäckt" potentiella problem i sitt nya system som du nu behöver hjälpa till att resonera om.

Nedan följer ett skärmdump från Hishams table of processes (**`htop`**) i terminalen. I **`htop`** visas bland annat processträdet och processernas tillstånd samt prioritet och ägare. Besvara följande frågor med informationen från htop i beaktning.

```
  PID USER      PRI  NI  VIRT   RES   SHR S CPU% MEM%   TIME+  Command
  770 root       20   0  344M  3016  2724 S  0.0  0.1  0:00.16 ├─ /usr/sbin/lightdm
 1026 root       20   0  224M  2880  2704 S  0.0  0.1  0:00.04 │  ├─ lightdm --session-child 12 19
 1260 kusin1     20   0 45776  3084  2236 S  0.0  0.2  0:00.66 │  │  ├─ /sbin/upstart --user
12730 kusin1     20   0  733M 54704 27812 S  0.0  2.7  3:03.78 │  │  │  ├─ /usr/lib/gnome-terminal-server
53546 kusin1     10 -10 27528  5836  3128 T  0.0  0.3  0:00.06 │  │  │  │  ├─ bash
53613 kusin1     20   0     0     0     0 Z  0.0  0.0  0:00.89 │  │  │  │  └─ gedit
```

**4(a)** Enligt din kusin är något "*fett skumt alltså*" med **`gedit`** för ingen sådan process kan hittas av kusin i det grafiska gränssnittet. När du googlar frasen ovan leds du bara till 1177.se för gasbesvär, men du tror att du förstår vad som åsyftas:

Processen GNOME text editor (**`gedit`**) använder till synes inga resurser: varken fysiskt eller virtuellt minne är allokerat och processorn nyttjas inte. Varför då?

**Skriv in ditt svar här**

gedit har just nu tillstånd "Z" vilket menas att vara zombie detta betyder att det är en process som inte ännu fått en stopp signal även fast den är klar. Vi ser att bash är i tillstånd "T" alltså pausat. Men gedit var fortfarande aktiv under detta så nu är gedit i zombie tillstånd då dess förelder aldrig skickade en stopp signal eftersom detn själv var plötsligt pausad. Jag vet inte riktigt vad detta har med gasbesvär och göra men gedit är ialla fall zombie.

Ord: 85

---

Besvarad.

**4(b)** Processen Bourne-Again SHell (**`bash`**) har ett prioritetsvärde som är lägre än övriga processer. Densamma gäller för nice-värdet. Prioriteten för processer i Linux beräknas utifrån följande formel:

$$PRI = 20 + NI$$

Din kusin är orolig att detta lägre värde kommer påverka användning av **`bash`**, uttryckt annorlunda "*att bash kommer kännas segt att köra*". Kusin vill nog gärna påstå att **`bash`** känts lite långsamt redan.

Förklara för din kusin vilken påverkan detta prioritetsvärde har för en process så som **`bash`**. Förklara även vad vi kan förvänta oss för praktisk effekt av prioritetsvärdet ovan under <u>både</u> vanlig användning av datorn och under extremfall.

**Skriv in ditt svar här**

En högre prioritet gör inte att programmet går snabbare. Det fungerar bara om det är "normal kö". Du kan prova att köra två single threaded windows. En med high priority och en med low. Du borde då observera att de kör lika fort eftersom att du har flera trådar.

Prioritet är bara det, en prioritet. Datorer jobbar idag med många trådar och en annorlunda prioritet ändrar inte hastigheten av det programmet då det körs när det körs.

Ord: 77

---

Besvarad.

**i** Anta att du har ett program vars uppgift är att iterativt inkrementera värdet ett hundra gånger i en global variabel vars initiala värde är noll, likt följande kodexempel:

```
x = 0
for i in range(100):
    x += 1
```

Istället för att addera värdena iterativt i en tråd har arbetet istället fördelats över 2 olika trådar där vardera tråd fått i uppgift att utföra 50 inkrementeringar på den globala variabeln. Detta med avsikten att fördela arbetet över två processorkärnor för att öka exekveringshastigheten. Vid exekveringen förväntar du dig ett deterministiskt utfall men får istället, förvirrande nog, olika utdata efter varje körning.

**5(a)** Vilka av följande värden, om något alls och oavsett sannolikhet, kan den globala variabeln erhålla efter en körning av det problematiska programmet?

**Välj ett eller flera alternativ:**

- [ ] -4
- [ ] 0
- [x] 1
- [x] 2
- [x] 3
- [x] 99
- [x] 100
- [ ] 101
- [ ] 200

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

---

Delvis rätt. 2 av 4 poäng.

**5(b)** Efter några justeringar till programkoden, där du bland annat implementerat mutex-lås för att skydda den kritiska sektionen så upptäcker du att utfallet nu blir deterministiskt med samma utdata efter varje körning. Du ändrar nu även datorsystemet till ett med fyra processorkärnor istället för två.

Vilka av följande värden, oavsett sannolikhet, kan den globala variabeln nu erhålla efter en körning av det justerade programmet?

**Välj ett eller flera alternativ:**

- [ ] -4
- [ ] 0
- [ ] 1
- [ ] 2
- [ ] 3
- [ ] 99
- [x] 100
- [ ] 101
- [ ] 200

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

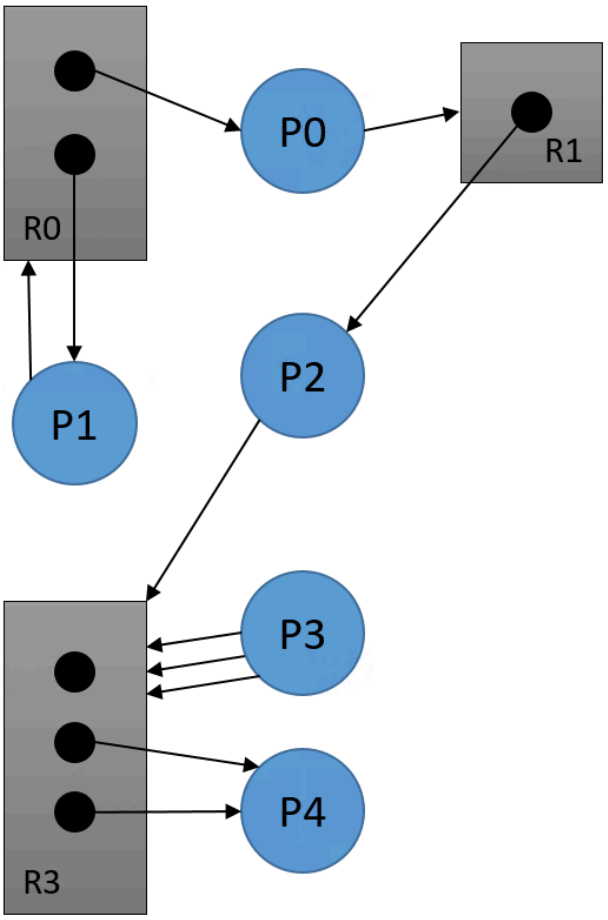
---

Rätt. 2 av 2 poäng.

**6** Deadlocks kan uppstå när vissa kriterier är uppfyllda. I ett visst datorsystem råder scenariot som avbildas i resursallokeringsgrafen nedan. Observera att:

- Systemet har ingen preemption eller deadlockförhindrande algoritm.
- I de situationer där flera processer efterfrågar samma resurs samtidigt allokeras resursen slumpartat.
- Processerna kan inte efterfråga fler resurser än de som redan representeras i grafen.
- Samtliga resursinstanser är odelbara.

Besvara följande frågor för det i grafen givna scenariot.




**6(a)** Har en deadlock uppstått? *Motivera ditt svar*.

**Skriv in ditt svar här**

Vi kan först räkna bort P4 eftersom P4 bara löser sig och försvinner direkt. Detta lämnar oss med 3 platser öppna i R3. Nu tävlar P3 och P2 om denna plats. Detta ser inte bra ut, då varken P3 eller P2 har en pil som pekar tillbaks ut så att de löser ut sig själva, detta menar att De sitter kvar i plats. Nu uppstår en deadlock då P3 väntar på P2 eller så väntar P2 på P3 i vilket fall löser blockerar de varandra och kan inte avsluta, en deadlock.

De som är direkt delaktiga i deadlocken är P2 och P3 men ifall P3 tar platserna före P2 så kommer varken P1 eller P0 kunna köra klart.

Ord: 118

Besvarad.

**6(b)** Resurs $R_3$ är en dyrbar resurs där varje aktiv instans ökar underhållskostnaderna för vår organisation. Vi önskar därför rationalisera bort så många instanser av $R_3$ som möjligt. Vi vill dock inte riskera att systemet hamnar i ett osäkert tillstånd (*unsafe state*) där en deadlock skulle kunna uppstå.

Utan att ändra övriga förutsättningar för scenariot, vad är det lägsta antalet instanser av $R_3$ som krävs för att systemet ska fungera (om än långsamt) samt vara i ett säkert tillstånd (*safe state*) och därmed helt undvika framtida deadlocks?

**Välj ett alternativ**

- 0
- 1
- 2
- 3
- 4
- 5
- Fler än 5 resursinstanser av $R_3$ krävs eftersom systemet för nuvarande är i ett unsafe state på grund av för få resursinstanser av $R_3$.
- Det spelar ingen roll hur många resursinstanser av $R_3$ vi har. Systemet är ändå i ett unsafe state och riskerar försättas i en deadlock.

---

Fel. 0 av 3 poäng.

**6(c)** Om en deadlock skulle kunna uppstå i det givna systemet, vilka processer skulle då vara delaktiga i deadlocken?

**Välj ett eller flera alternativ:**

- [x] $P_0$
- [x] $P_1$
- [x] $P_2$
- [x] $P_3$
- [ ] $P_4$
- [ ] Inga processer. En deadlock kan inte uppstå i det givna systemet.

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

---

Delvis rätt. 0 av 3 poäng.

CPU-schemaläggning används för att bestämma vilken process, eller när stöd finns, vilken tråd som ska exekvera på processorn. Trådar ankommer kontinuerligt till en redokö efter ett event eller avslutad I/O-operation.

I följande scenario, vid tidpunkten $t$ har processorn precis blivit tillgänglig. Två trådar har också precis ankommit till redokön i väntan på att bli schemalagda. Med ytterst avancerat estimerande (en spåkula) kan vi även se att fler trådar snart ankommer till kön, enligt nedan angivna ankomsttider.

Tabell. Redokön vid tidpunkten $t$.

| Köplats (#) | Tråd | Uppskattad CPU-sprint (ms) | Ankomsttid (ms) |
|---|---|---|---|
| 1 | $T_1$ | 10 | $t$ |
| 2 | $T_2$ | 4 | $t$ |
| 3 | $T_3$ | 10 | $t$ |
| -- | $T_4$ | 6 | $t+10$ |

Du kan för dessa uppgifter anta att tidskostnaden för context switching är försumbar. Det är enklast att visualisera sig scenariot om du antar att $t = 0$ vid starten.

**7(a)** Anta att trådarna schemaläggs med *First-come, First-served (FCFS)*. Vad blir väntetiden för varje tråd och medelväntetiden (tiden varje tråd väntar i snitt)? *Svara exakt*.

T1: 4 ms.

T2: 0 ms.

T3: 14 ms.

T4: 14 ms.

Medelväntetid: 8 ms.

Delvis rätt. 2 av 5 poäng.

**7(b)** Anta att trådarna schemaläggs med *Round Robin (RR)* där tidskvantumet är $q = 4\ ms$. Vad blir väntetiden för varje tråd och medelväntetiden (tiden varje tråd väntar i snitt)? *Svara exakt*.

T1: 16 ms.

T2: 4 ms.

T3: 22 ms.

T4: 26 ms.

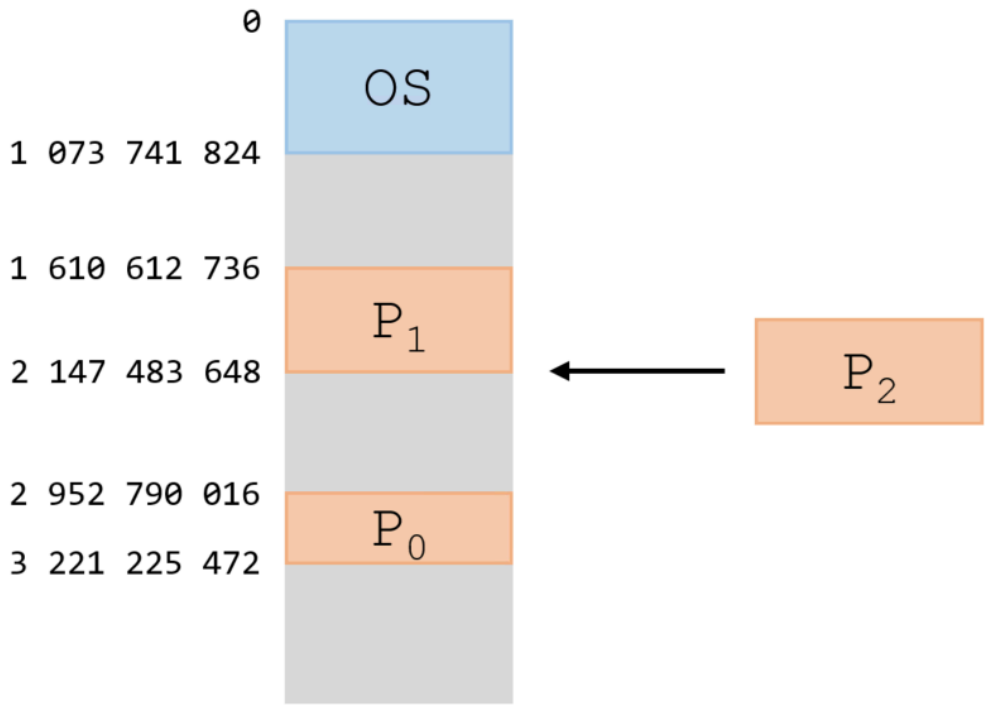
Medelväntetid: 10,5 ms.

Delvis rätt. 2 av 5 poäng.

Sammanhängande allokering kan användas för att allokera processers minne i enhetliga minnesspann.

Anta att du har ett system med 4 GiB primärminne som allokeras sammanhängande. Operativsystemet och två processer har redan allokerats till minnet i detta system, se figuren nedan. I figuren anges adressen för starten av varje block av minne på decimalform. T.ex. är operativsystemet placerat i spannet 0 till och med 1 073 741 823. Observera att figuren inte är skalenlig.

Utrymme behöver nu allokeras för en ny process $P_2$ som kräver 500 MiB. Besvara följande frågor relaterade till det givna scenariot.




**8(a)** På vilken adress kommer processens bas placeras om metoden **first-fit** används? *Svara med talbas 10*.

Ange adressen här: 1073741824 .

Rätt. 2 av 2 poäng.

**8(b)** På vilken adress kommer processens bas placeras om metoden **best-fit** används? *Svara med talbas 10*.

Ange adressen här: 1073741824 .

Rätt. 2 av 2 poäng.

**8(c)** På vilken adress kommer processens bas placeras om metoden **worst-fit** används? *Svara med talbas 10*.

Ange adressen här: 3221225472 .

Rätt. 2 av 2 poäng.

**8(d)** Anta nu att du skapar ytterligare processer som alla har samma storlek som $P_2$. Du upprepar detta tills att det inte längre går att allokera minne till nya sådana processer. Enligt 50-procentregeln, hur mycket minne har nu gått förlorat till extern fragmentering?

*Anta att inga processer stängs av och att ingen kompaktering eller swapping sker. Svara i MiB*.

Extern fragmentering: 1365 MiB.

---

Rätt. 2 av 2 poäng.

**8(e)** 50-procentregeln är endast en tumregel. Hur mycket minne har egentligen gått förlorat till extern fragmentering?

Extern fragmentering: 1104 MiB.

---

Fel. 0 av 2 poäng.

**9(a)** Anta att du har ett filsystem som använder länkad allokering med 64-bitars adresser och en blockstorlek på 64 B.

Vad blir den maximala filstorleken och overheaden för en fil i detta system? *Svaren anges i EiB*.

Maximal filstorlek: 1024 EiB.

Overhead: 128 EiB.

Tips:

- $1\ EiB = 1024\ PiB$
- $1\ PiB = 1024\ TiB$

---

Rätt. 4 av 4 poäng.

**9(b)** Anta att du har ett filsystem liknande UFS som använder hierarkisk indexerad allokering, även kallat *combined scheme*, med 32-bitars adresser och en blockstorlek på 64 B. I inoden finns 10 direkta, 1 indirekt och 1 dubbelt indirekt pekare som leder till data- och indexblock.

Vad blir den maximala filstorleken och overheaden i detta system? *Svaren anges i bytes.*

Maximal filstorlek: 67200 B.

Overhead: 1119 B.

---

Fel. 0 av 4 poäng.

**9(c)** Du arbetar nu med det tidigare angivna filsystemet med hierarkisk indexerad allokering. Anta att du har möjligheten att ändra adresstorleken. Blockstorleken och övriga egenskaper är oförändrade. Vilken adresstorlek $a$ skulle ge den största möjliga maximala filstorleken?

Du kan anta att adresstorleken $a$ är ett positivt antal bitar: $a \in \mathbb{Z}^+$ som inte nödvändigtvis är en tvåpotens.

*Motivera ditt svar*.

**Skriv in dina beräkningar och ditt svar här**

Som det går att se under beräkningar så ser vi att det slutgiltiga a är större ju mindre adresstorleken blir. Därför får vi tekniskt sett störst adresstorlek om vi har adresstorleken 1. Detta skulle dock vara opraktiskt.

10*64 + ( 1 * 64/(32/8) ) * 64 + (1 * 64 / 32 / 8)^2 * 64

Ord: 56

---

Besvarad.